

SIMATS ENGINEERING

ASSESSMENT TOOL 1: AI Model Design Exercise

COURSE NAME: ARTIFICIAL INTELLIGENCE COURSE CODE: MLA01
AND EXPERT SYSTEMS

COURSE OUTCOME COVERED

CO4: *Develop intelligent software agents using suitable agent architectures and learning techniques.*(BTL 3)

Assessment Tool Weightage: 40 %

Total Marks: 100

Time: 90 Mins

No. of Questions: 1

Annexure A AI Model Design Exercise

Code of Conduct:

/ P Surendra Kumar Reddy (192425224) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:*P Surendra Kumar Reddy*

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Dataset Selection and Data Understanding	10%	
3. Data Preprocessing and Feature Engineering	10%	
4. AI Technique Selection and Justification	10%	
5. Model Design / Architecture	10%	
6. Algorithm / Pseudocode Development	5%	
7. Model Implementation and GitHub Repository	15%	
8. Experimental Design and Results	10%	
9. Performance Evaluation and Metrics	10%	
10. Result Analysis and Interpretation	5%	
11. Limitations and Future Enhancements	5%	
12. Documentation and Technical Presentation	10%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

ANSWER ANY ONE QUESTION

S.No.	Scenario / Problem Statement	AI Model / Technique to be Developed	Expected Development Outcome
1	Smart Loan Approval: A bank wants to automatically determine whether a customer is eligible for a loan using income, credit score, employment status, loan amount, and repayment history.	Decision Tree	Design and implement a decision-tree-based AI model and determine the most suitable attribute-selection measure such as Information Gain, Gain Ratio, or Gini Index.
2	Student Performance Prediction: A university wants to predict whether a student is likely to pass or fail a course using attendance, internal marks, assignment performance, study hours, and previous academic performance.	Inductive Learning	Construct a predictive model from training examples and derive generalized decision rules from the learned patterns.
3	Disease Risk Prediction: A healthcare system needs to classify patients into low-risk and high-risk categories based on clinical measurements.	Statistical Learning	Design a statistical learning model, train it using patient data, and evaluate its generalization ability on unseen data.
4	Image Recognition System: An organization wants to automatically classify images into categories such as vehicles, animals, and buildings.	Neural Network	Design and implement a neural-network classifier by specifying the network architecture, activation functions, loss function, optimizer, and training procedure.
5	Handwritten Character Recognition: A system must recognize handwritten characters from scanned images.	Multilayer Neural Network	Develop a multilayer neural network and demonstrate how backpropagation updates weights to minimize classification error.

6	Non-Linear Classification: A dataset contains two classes that cannot be separated using a straight-line decision boundary.	SVM with Kernel Method	Design an SVM-based classifier using an appropriate kernel function and justify the selected kernel based on the characteristics of the dataset.
7	Intelligent Recommendation Agent: An online platform wants an agent that learns which recommendations to provide based on user responses.	Reinforcement Learning	Design an RL model by defining the states, actions, rewards, policy, and learning strategy, and demonstrate how the agent learns from user feedback.
8	Autonomous Game Agent: An AI agent must learn to play a game where successful actions receive positive rewards and unsuccessful actions receive penalties.	Reinforcement Learning	Develop an RL-based game agent and demonstrate how its policy and performance improve through repeated interaction with the environment.
9	Explainable Diagnosis System: A medical AI system predicts a disease but must also provide an explanation for its prediction.	Explanation-Based Learning	Design an explanation-based learning approach using domain knowledge and training examples to generate an understandable justification for the prediction.
10	AI Model Selection Challenge: A company provides a dataset for predicting customer churn.	Comparative AI Model Design	Design and compare at least two learning approaches from decision trees, statistical learning, neural networks, or kernel methods. Select the best model based on accuracy, interpretability, computational requirements, and generalization ability.

Structure of the Report:

S.No.	Report Component	What the Student Should Include
1	Problem Statement	Clearly describe the real-world problem, input data, expected output, and AI task to be solved.
2	Objectives	State the specific goals of developing the AI model and expected outcomes.
3	Dataset Description	Provide dataset source, number of instances, features, target variable, data types, and important attributes.
4	Data Preprocessing	Explain data cleaning, missing-value handling, encoding, normalization/scaling, feature selection, and train-test splitting.
5	Selected AI Technique and Justification	Identify the selected AI technique and justify its suitability for the problem.
6	Model Design / Architecture	Present the model structure, components, input/output, parameters, hyperparameters, and architecture/flow diagram.
7	Algorithm / Pseudocode	Provide the algorithmic steps or pseudocode showing model training and prediction/decision-making.
8	Implementation	Provide programming language, libraries/tools, important code sections, and implementation details.
9	Experimental Results	Present results using tables, graphs, sample predictions, learning curves, confusion matrices, or other relevant outputs.
10	Performance Evaluation	Evaluate the model using appropriate performance metrics such as accuracy, precision, recall, F1-score, MSE, RMSE, or cumulative reward.
11	Result Analysis	Interpret results, identify important patterns, compare actual and predicted outcomes, and discuss strengths and weaknesses.
12	Limitations	Discuss limitations related to dataset, model complexity, computational resources, accuracy, generalization, or deployment.
13	Future Enhancements	Suggest improvements such as additional data, feature engineering, hyperparameter tuning, alternative algorithms, or deployment.
14	Conclusion	Summarize the problem, methodology, major findings, model performance, and overall effectiveness.
15	References	List datasets, research papers, books, websites, documentation, libraries, and other sources used.
16	GitHub Repository Link	Provide the public GitHub repository URL containing the complete source code, dataset information/source, implementation, results, README, and supporting files.

Q1. Smart Loan Approval Using Decision Tree

1. Problem Statement

Loan approval is an important decision-making process in the banking sector. Traditionally, loan applications may require manual evaluation of financial and employment information, which can be time-consuming and may lead to inconsistent decisions.

The objective of this project is to develop an Artificial Intelligence-based **Smart Loan Approval System** that predicts whether a customer's loan application should be **Approved** or **Rejected**.

The system uses customer attributes such as:

- Income
- Credit Score
- Employment Status
- Loan Amount
- Repayment History

A **Decision Tree Classification algorithm** is used to learn patterns from the customer data and predict the loan approval status.

2. Objectives

The main objectives of the proposed system are:

1. To develop an AI-based loan approval prediction system.
2. To analyze customer financial and employment information.
3. To preprocess and encode the dataset for machine learning.
4. To implement a Decision Tree classification algorithm.
5. To use the **Gini Index** for decision-node selection.
6. To divide the dataset into training and testing sets.
7. To predict loan approval for new customers.
8. To evaluate the model using accuracy, precision, recall and F1-score.
9. To analyze the confusion matrix.
10. To identify the most important features influencing loan approval.

3. Dataset Description

Dataset Source: The dataset was synthetically generated using Python and NumPy specifically for this AI Model Design Exercise. It contains 100 customer records and was designed to represent the financial and employment characteristics required for loan approval prediction. Since the dataset is synthetic, it does not originate from a real banking institution.

The dataset contains **100 customer records and 6 attributes**.

Dataset Attributes

Attribute	Description	Type
Income	Annual income of the customer	Numerical
CreditScore	Credit score of the customer	Numerical
EmploymentStatus	Employment condition of the customer	Categorical
LoanAmount	Amount of loan requested	Numerical
RepaymentHistory	Previous repayment behavior	Categorical
LoanApproval	Loan approval decision	Target

Target Variable

The target variable is **LoanApproval**, with two possible classes:

- Approved
- Rejected

Dataset Distribution

Loan Approval	Number of Records
Approved	76
Rejected	24
Total	100

Dataset Properties

- Total records: **100**
- Total columns: **6**
- Input features: **5**
- Target variable: **1**
- Missing values: **0**

- Duplicate records: 0

```

First 10 Records:
   Income  CreditScore  EmploymentStatus  LoanAmount  RepaymentHistory  LoanApproval
0   35795         561         Employed      365656         Average      Rejected
1   20860         715      Self-Employed      363817           Good      Approved
2   96820         792      Self-Employed      616309           Good      Approved
3   74886         598         Employed      659321         Average      Rejected
4   26265         671      Self-Employed      565890         Average      Rejected
5   57194         713         Employed      693415           Good      Approved
6   64131         534      Self-Employed      645101           Good      Rejected
7   80263         726      Self-Employed      236631           Good      Approved
8   36023         600         Employed      442289         Average      Rejected
9   61090         630         Employed      272991         Average      Approved

Dataset Shape:
(100, 6)

Column Names:
['Income', 'CreditScore', 'EmploymentStatus', 'LoanAmount', 'RepaymentHistory', 'LoanApproval']

Dataset Information:
<class 'pandas.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 6 columns):
#   Column                Non-Null Count  Dtype
---  ---                ---
0   Income                 100 non-null   int32
1   CreditScore            100 non-null   int32
2   EmploymentStatus       100 non-null   str
3   LoanAmount             100 non-null   int32
4   RepaymentHistory       100 non-null   str
5   LoanApproval           100 non-null   str
dtypes: int32(3), str(3)
memory usage: 3.6 KB

```

Figure 1: Dataset Overview of the Smart Loan Approval Dataset

4. Data Preprocessing and Feature Engineering

Data preprocessing was performed before training the Decision Tree model.

4.1 Missing Value Checking

The dataset was checked for missing values. All six attributes contained **0 missing values**.

Therefore, no missing-value replacement was required.

4.2 Duplicate Checking

The dataset was also checked for duplicate records.

Number of duplicate records = 0

Therefore, no duplicate records were removed.

4.3 Categorical Encoding

Categorical attributes were converted into numerical values using Label Encoding.

Employment Status

Category	Encoded Value
Employed	0
Self-Employed	1
Unemployed	2

Repayment History

Category	Encoded Value
Average	0
Good	1
Poor	2

Loan Approval

Category	Encoded Value
Approved	0
Rejected	1

4.4 Feature Selection

The five input features used by the model were:

- Income
- CreditScore
- EmploymentStatus
- LoanAmount
- RepaymentHistory

The target variable was:

 LoanApproval

4.5 Train-Test Split

The dataset was divided using an 80:20 ratio:

- Training samples = **80**
- Testing samples = **20**

This allowed the model to learn from the training data and evaluate its performance on unseen testing data.

5. Selected AI Technique and Justification

Decision Tree Classification

A **Decision Tree Classifier** was selected for the Smart Loan Approval problem.

The problem is a classification problem because the output belongs to one of two categories:

Approved / Rejected

The Decision Tree creates a sequence of decision rules based on the input features and produces the final loan approval decision at the leaf node.

Gini Index

The model uses the **Gini Index** as the splitting criterion.

The Gini Index measures the impurity of a node:

$$\text{Gini} = 1 - \sum p_i^2$$

where p_i represents the probability of a sample belonging to a particular class.

A lower Gini value represents a purer node.

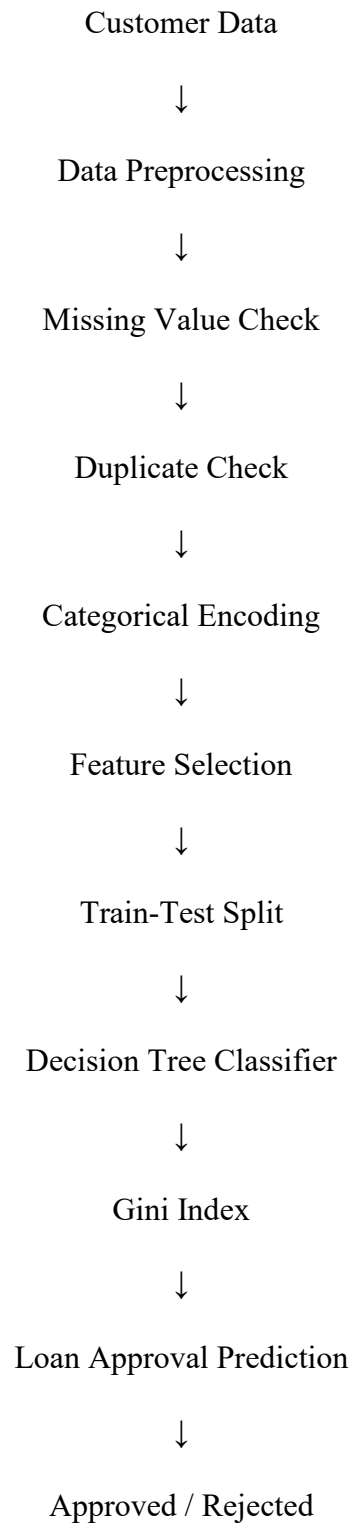
Why Decision Tree?

The Decision Tree was selected because:

- It is suitable for classification problems.
- It is easy to understand and interpret.
- It produces rule-based decisions.
- It can handle numerical and encoded categorical features.
- Its structure can be visualized.
- Feature importance can be extracted from the model.

6. Model Design / Architecture

The overall architecture of the system is:



Model Parameters

Parameter	Value
Algorithm	Decision Tree
Criterion	Gini Index
Maximum Depth	5
Minimum Samples Split	4
Minimum Samples Leaf	2

Random State	42
Training Samples	80
Testing Samples	20

Decision Tree Visualization

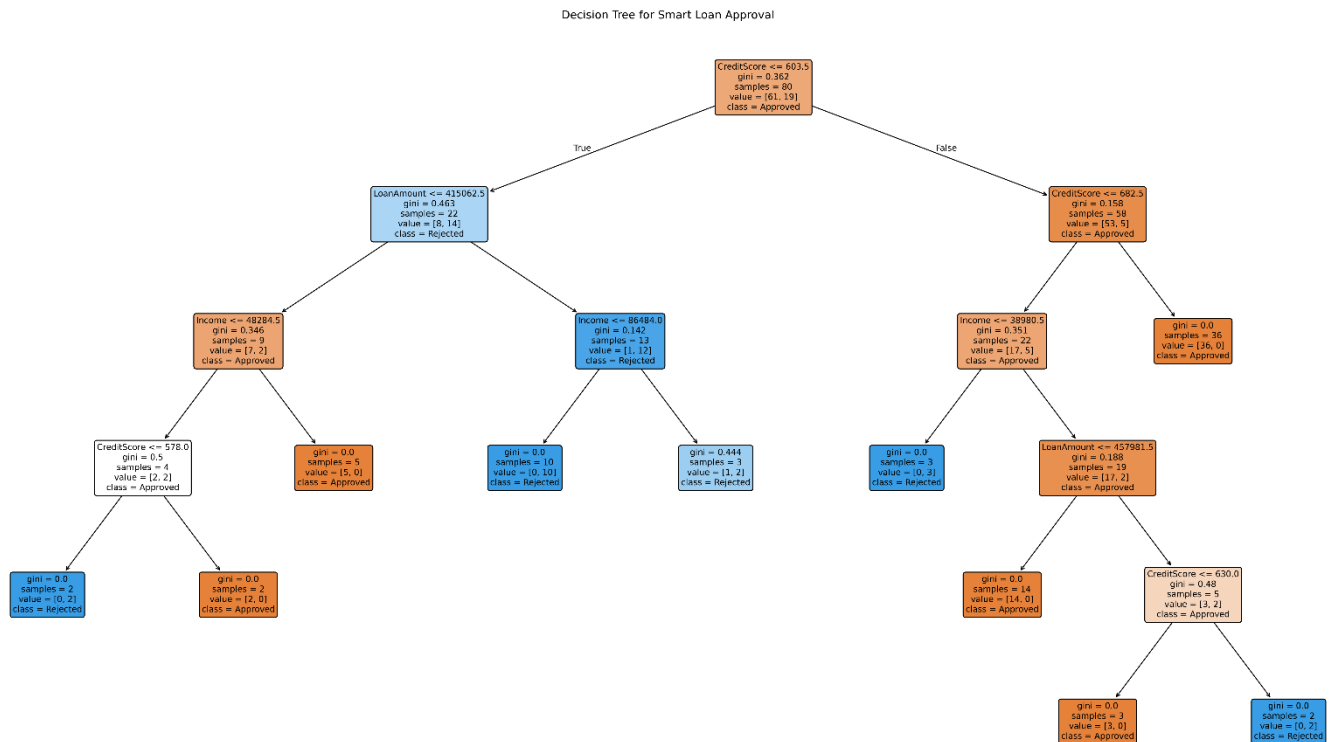


Figure 2: Decision Tree Model for Smart Loan Approval

Interpretation

The generated Decision Tree shows that **CreditScore** is used at the root node, indicating that it is the most influential initial splitting feature. The subsequent branches use features such as **LoanAmount**, **Income** and **CreditScore** to classify customers as Approved or Rejected.

The tree uses Gini impurity at its decision nodes and produces Approved or Rejected classes at the leaf nodes.

7. Algorithm / Pseudocode

Algorithm: Smart Loan Approval

START

1. Import required Python libraries:

- Pandas

- NumPy
- Scikit-learn
- Matplotlib
- Seaborn

2. Generate/load the loan approval dataset.

3. Store the dataset in a DataFrame.

4. Display the first 10 records.

5. Check the dataset:

- Display dataset shape
- Display column names
- Display data information
- Check missing values
- Check duplicate records
- Display class distribution

6. Preprocess the data:

Encode EmploymentStatus:

Employed = 0

Self-Employed = 1

Unemployed = 2

Encode RepaymentHistory:

Average = 0

Good = 1

Poor = 2

Encode LoanApproval:

Approved = 0

Rejected = 1

7. Separate the input features and target:

Features =

- Income
- CreditScore
- EmploymentStatus
- LoanAmount
- RepaymentHistory
- Target = LoanApproval

8. Split the dataset into:

- ✓ Training data = 80%
- ✓ Testing data = 20%

9. Create a Decision Tree Classifier.

10. Set the model parameters:

- Criterion = Gini Index
- Maximum Depth = 5
- Minimum Samples Split = 4
- Minimum Samples Leaf = 2
- Random State = 42

11. Train the Decision Tree using the training data.

12. Use the trained model to predict loan approval

for the testing data.

13. Evaluate the model using:

 Accuracy

 Precision

 Recall

 F1-score

 Classification Report

Confusion Matrix

14. Generate the confusion matrix

and save it as `confusion_matrix.png`.

15. Calculate feature importance for:

- Income
- CreditScore
- EmploymentStatus
- LoanAmount
- RepaymentHistory

16. Generate and save the feature importance graph

as `feature_importance.png`.

17. Generate and save the Decision Tree visualization

as `decision_tree.png`.

18. Provide a new customer's details:

- Income = 65000
- CreditScore = 750
- EmploymentStatus = Employed
- LoanAmount = 400000
- RepaymentHistory = Good

19. Apply the same encoding used during training.

20. Use the trained Decision Tree to predict

the new customer's loan approval.

21. Display:

- Predicted class
- Approval probability
- Rejection probability

22. Save prediction results and feature importance

data into CSV files.

END

8. Implementation

The system was implemented using **Python**.

Technologies Used

- Python
- Pandas
- NumPy
- Scikit-learn
- Matplotlib
- Seaborn

Implementation Steps

1. Generate the customer dataset.
2. Display the first 10 records.
3. Check dataset dimensions.
4. Display column names and data information.
5. Check class distribution.
6. Check missing values.
7. Check duplicate records.
8. Encode categorical attributes.
9. Separate features and target.
10. Split data into training and testing sets.
11. Create and train the Decision Tree.
12. Generate predictions.
13. Calculate performance metrics.
14. Generate the confusion matrix.
15. Generate the Decision Tree visualization.
16. Generate feature importance.
17. Predict the loan status of a new customer.
18. Save prediction and feature-importance results.

Generated Files

Your implementation successfully generated:

- loan_prediction.py
- confusion_matrix.png
- decision_tree.png
- feature_importance.png
- feature_importance.csv
- prediction_results.csv

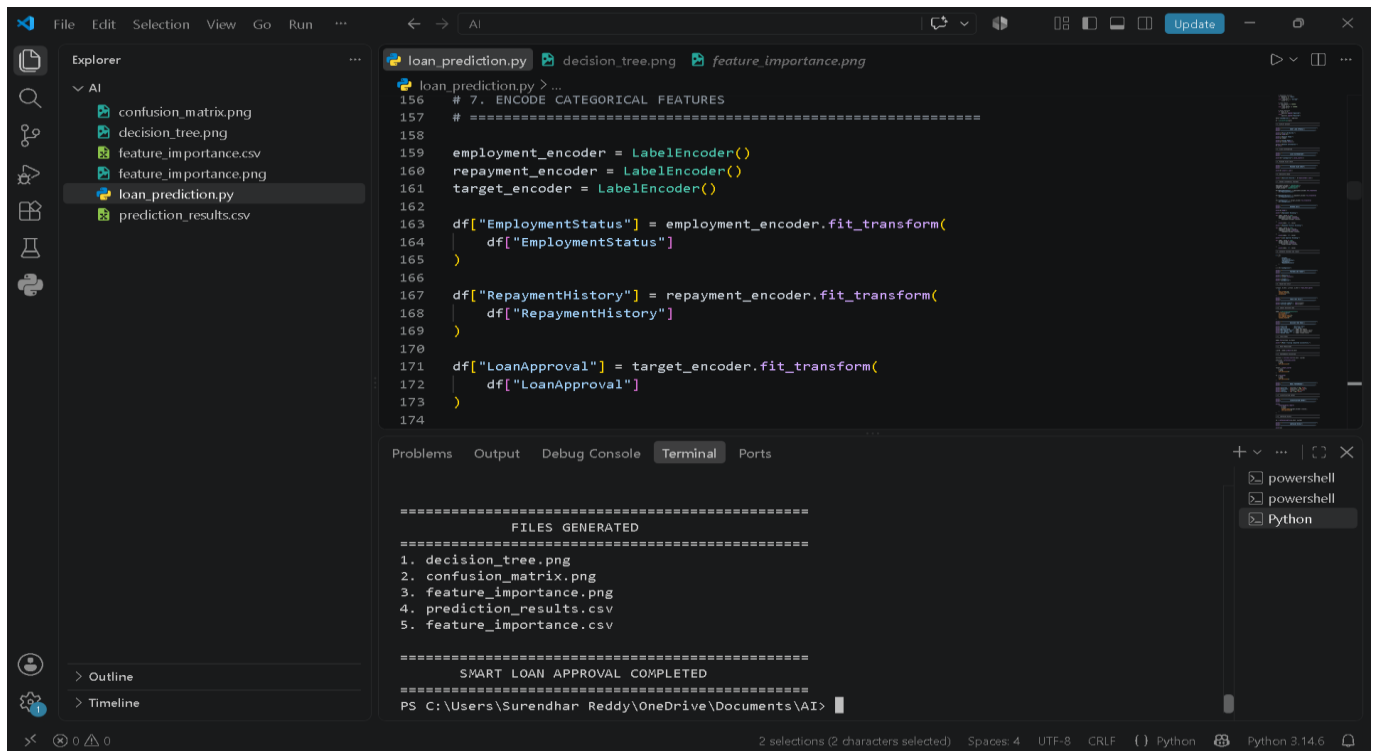


Figure 3: Python Implementation and Generated Results

9. Experimental Results

The trained Decision Tree was evaluated using the **20 testing samples**.

Performance Results

Metric	Result
Accuracy	75.00%
Precision	50.00%
Recall	60.00%
F1-Score	54.55%

Classification Report

Class	Precision	Recall	F1-Score	Support
Approved	0.86	0.80	0.83	15
Rejected	0.50	0.60	0.55	5
Overall Accuracy				75%

9.1 New Customer Prediction

The trained Decision Tree model was also tested with a new customer to demonstrate its prediction capability. The following customer information was provided to the model:

Attribute	Value
Income	65,000
CreditScore	750
EmploymentStatus	Employed
LoanAmount	400,000
RepaymentHistory	Good

The model predicted the loan application as:

Loan Approval Prediction: Approved

The prediction probabilities were:

Prediction	Probability
Approved	100.00%
Rejected	0.00%

This result demonstrates that the trained Decision Tree can use the applicant's financial and repayment information to generate a loan approval prediction for a new customer.

10. Confusion Matrix Analysis

The actual confusion matrix produced by your implementation is:

Actual / Predicted	Approved	Rejected
Approved	12	3
Rejected	2	3

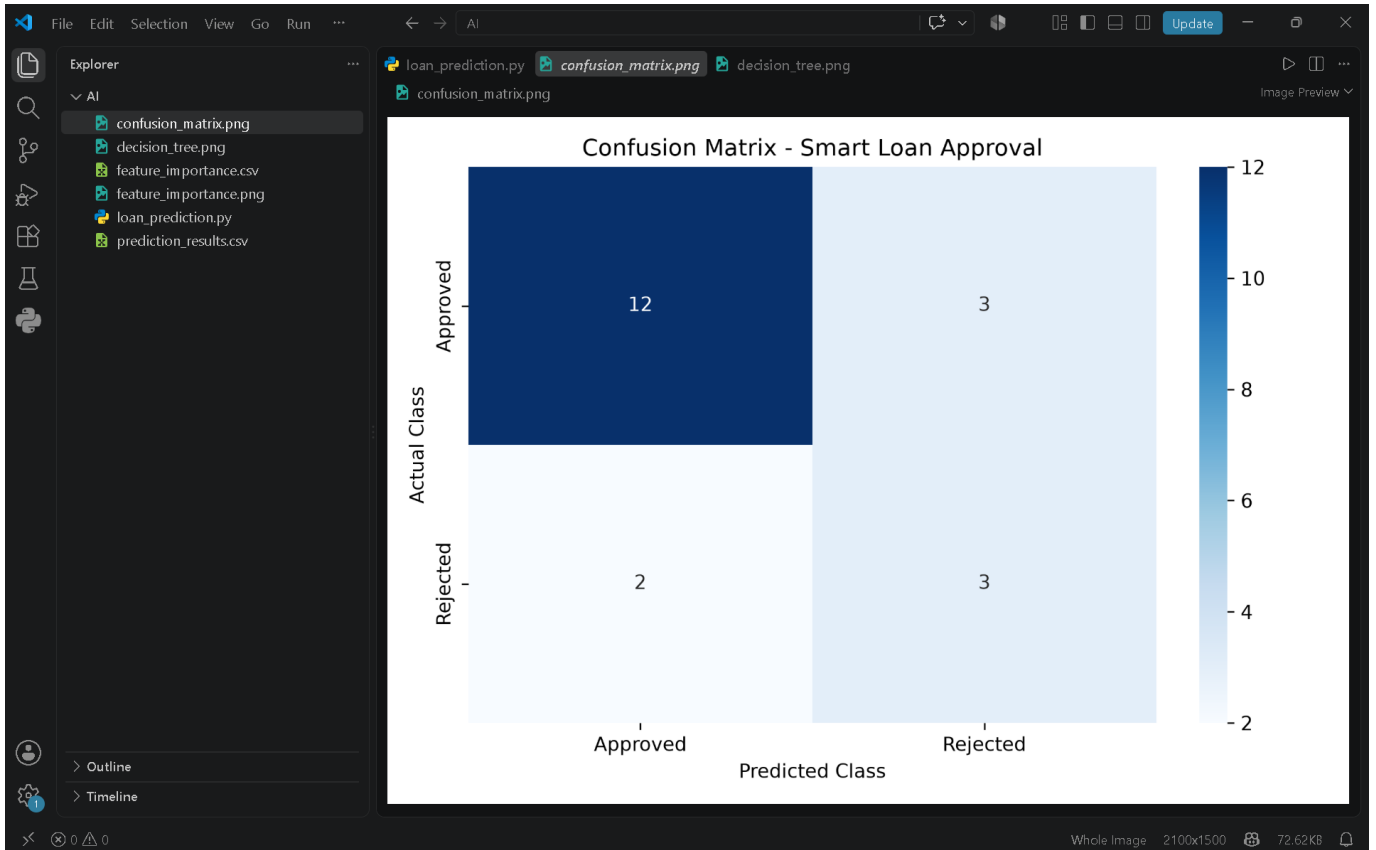


Figure 4: Confusion Matrix of Smart Loan Approval Model

Interpretation

The confusion matrix shows:

- **12** Approved applications were correctly predicted as Approved.
- **3** Approved applications were incorrectly predicted as Rejected.
- **2** Rejected applications were incorrectly predicted as Approved.
- **3** Rejected applications were correctly predicted as Rejected.

Therefore, the model correctly classified:

$12 + 3 = 15$ out of 20 test samples.

This corresponds to the observed **75% accuracy**.

11. Feature Importance Analysis

The Decision Tree was also used to determine the relative importance of the input features.

Feature Importance

The generated feature-importance graph shows the following approximate ranking:

Rank	Feature	Approx. Importance
1	CreditScore	0.56
2	LoanAmount	0.23
3	Income	0.21
4	EmploymentStatus	~0.00
5	RepaymentHistory	~0.00

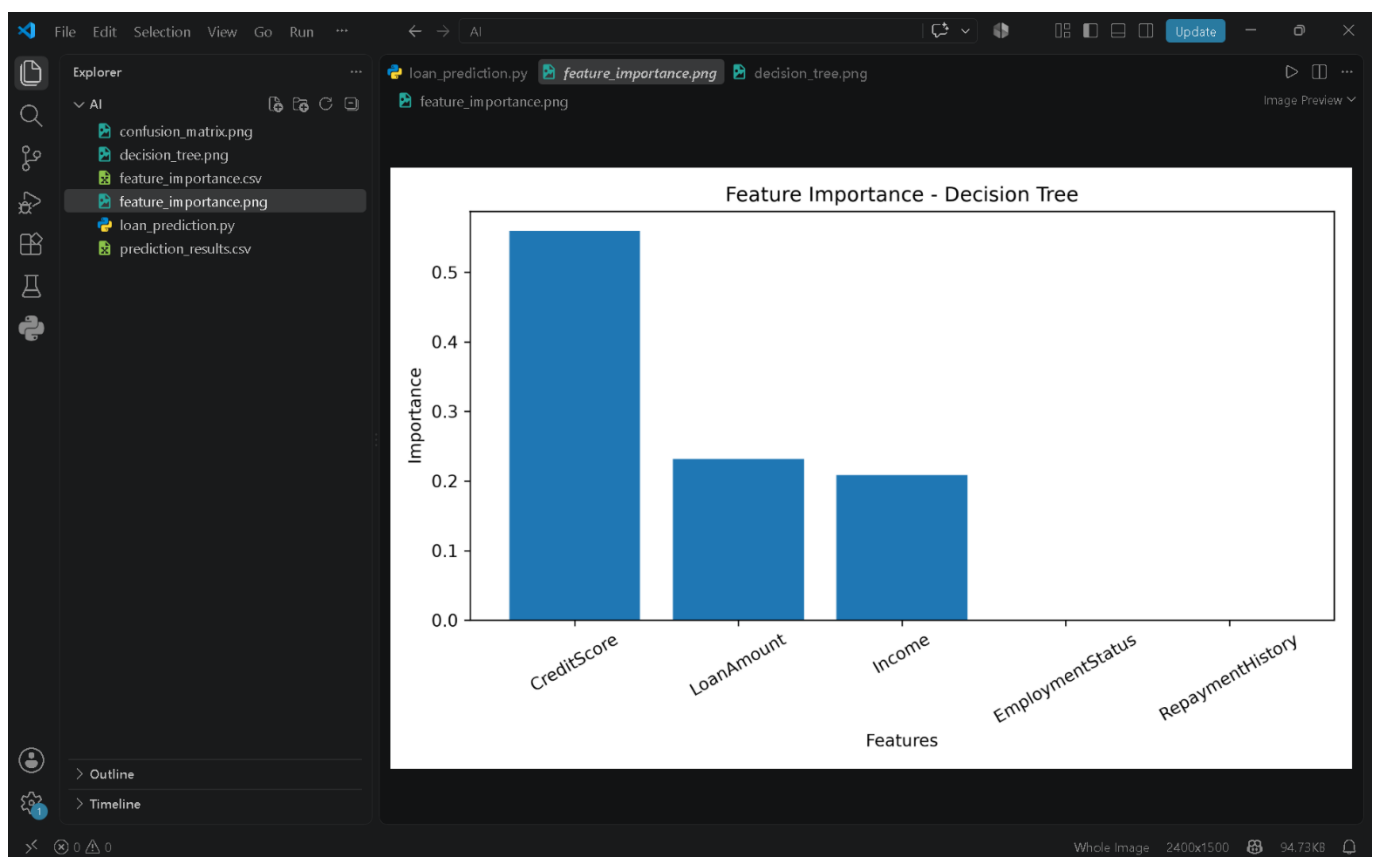


Figure 5: Feature Importance of Decision Tree

Interpretation

The feature-importance analysis indicates that **CreditScore is the most influential feature** in the trained Decision Tree model.

CreditScore contributes approximately **0.56** of the total feature importance, followed by LoanAmount at approximately **0.23** and Income at approximately **0.21**.

EmploymentStatus and RepaymentHistory have negligible importance in this particular trained model.

This result is consistent with the Decision Tree visualization, where **CreditScore** appears at the root node.

12. Performance Evaluation

The Decision Tree model achieved an overall **accuracy of 75%** on the testing dataset.

Accuracy

The model correctly classified **15 out of 20** testing samples.

Therefore:

$$\text{Accuracy} = 75\%$$

Precision

The overall precision was **50%**.

This indicates that, for the positive class used in the model's evaluation, **50% of the predicted positive cases were correctly classified.**

Recall

The overall recall was **60%**.

This indicates that the model correctly identified **60% of the actual positive cases** for the evaluated positive class.

F1-Score

The overall F1-score was **54.55%**, representing the balance between precision and recall.

Class-wise Performance

The model performed better for the **Approved** class:

- Precision = **86%**
- Recall = **80%**
- F1-score = **83%**

For the **Rejected** class:

- Precision = **50%**
- Recall = **60%**
- F1-score = **55%**

Therefore, the model is better at identifying **Approved** applications than **Rejected** applications.

13. Result Analysis and Interpretation

The experimental results demonstrate that the Decision Tree can classify loan applications based on customer financial and employment information.

The model achieved **75% accuracy** on the testing dataset.

The Decision Tree visualization shows **CreditScore as the root decision feature**. The feature-importance analysis also identifies CreditScore as the most important feature, with an importance of approximately **0.56**.

This consistency between the Decision Tree structure and feature-importance graph indicates that credit score plays a major role in the model's decision-making process.

The confusion matrix shows that 15 out of 20 test cases were classified correctly. However, five test cases were incorrectly classified.

The model's performance on the Approved class was stronger than its performance on the Rejected class. This indicates that the model has more difficulty distinguishing rejected loan applications.

One possible reason is the relatively small dataset containing only **100 records**, with **76 Approved** and **24 Rejected** applications. A larger and more diverse dataset could provide more information for learning the characteristics of rejected applications.

14. Limitations

The proposed system has the following limitations:

1. The dataset contains only 100 records.
2. The dataset is synthetically generated.
3. The number of Approved and Rejected records is not equal.
4. The model may not generalize to real-world banking data.
5. Only five input features are considered.

6. Real loan approval decisions may require additional financial and personal information.
7. The Decision Tree may be sensitive to changes in the training dataset.
8. The current model requires further validation before real-world deployment.

15. Future Enhancements

The system can be improved in the future by:

1. Using a larger real-world loan dataset.
2. Adding more relevant financial attributes.
3. Applying cross-validation.
4. Performing hyperparameter optimization.
5. Comparing Decision Tree with Random Forest, Logistic Regression and other classification algorithms.
6. Addressing class imbalance.
7. Developing a web-based loan approval interface.
8. Adding explainable AI techniques to show why a loan was approved or rejected.
9. Continuously retraining the model with new data.
10. Deploying the trained model as a real-time prediction service.

16. Conclusion

The Smart Loan Approval system was successfully developed using a **Decision Tree classification algorithm**. The model was trained using applicant information such as **Income, Credit Score, Employment Status, Loan Amount, and Repayment History** to predict whether a loan application would be **Approved or Rejected**.

The dataset contained **100 records**, with **80 samples used for training** and **20 samples used for testing**. The Decision Tree model achieved an overall accuracy of 75% on the testing dataset. For the Rejected class, the model achieved 50% precision, 60% recall, and 54.55% F1-score. The class-wise evaluation showed that the model performed better for the **Approved** class, achieving **86% precision, 80% recall, and 83% F1-score**. For the **Rejected** class, the model achieved **50% precision, 60% recall, and 55% F1-score**. Therefore, the model was more effective at identifying approved applications than rejected applications.

The feature importance analysis showed that **Credit Score** was the most important feature, followed by **Loan Amount** and **Income**. Employment Status and Repayment History had zero feature importance in the trained Decision Tree model.

Finally, when tested with a new customer having an income of **65,000**, credit score of **750**, loan amount of **400,000**, employed status, and good repayment history, the system predicted the loan as **Approved**.

with a prediction probability of **100%**.

Overall, the project demonstrates how machine learning can be applied to automate loan approval prediction and assist in making data-driven lending decisions.

REFERENCES

1. Scikit-learn, "Decision Trees," Scikit-learn Documentation.
<https://scikit-learn.org/stable/modules/tree.html>
2. Scikit-learn, "DecisionTreeClassifier," Scikit-learn Documentation.
<https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html>
3. Scikit-learn, "Classification Metrics," Scikit-learn Documentation.
<https://scikit-learn.org/stable/api/sklearn.metrics.html>
4. Scikit-learn, "classification_report," Scikit-learn Documentation.
https://scikit-learn.org/stable/modules/generated/sklearn.metrics.classification_report.html
5. Scikit-learn, "precision_score," Scikit-learn Documentation.
https://scikit-learn.org/stable/modules/generated/sklearn.metrics.precision_score.html
6. Scikit-learn, "recall_score," Scikit-learn Documentation.
https://scikit-learn.org/stable/modules/generated/sklearn.metrics.recall_score.html
7. Scikit-learn, "f1_score," Scikit-learn Documentation.
https://scikit-learn.org/stable/modules/generated/sklearn.metrics.f1_score.html
8. Python Software Foundation, "Python Documentation."
<https://docs.python.org/3/>
9. pandas Development Team, "pandas Documentation."
<https://pandas.pydata.org/docs/>
10. Matplotlib Development Team, "Matplotlib Documentation."
<https://matplotlib.org/stable/>

GitHub Repository Link:

URL: <https://github.com/suri307/MLA0103-AI/tree/main/CO4>

Evaluation Rubrics:

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Problem Analysis and Understanding	10%	9–10% – Clearly analyzes the real-world problem, objectives, inputs, outputs, constraints, and expected AI outcome.	7–8% – Understands the problem and objectives with minor omissions.	5–6% – Shows partial understanding with some important elements missing.	0–4% – Problem is poorly understood or incorrectly defined.
2. Dataset Selection and Understanding	10%	9–10% – Selects a highly relevant dataset and clearly explains features, target, source, size, and characteristics.	7–8% – Appropriate dataset with adequate description and minor omissions.	5–6% – Dataset is usable but description or relevance is limited.	0–4% – Dataset is inappropriate, poorly described, or missing.
3. Data Preprocessing and Feature Engineering	10%	9–10% – Performs appropriate cleaning, transformation, encoding, scaling, feature selection, and data splitting with clear justification.	7–8% – Performs most required preprocessing correctly with minor issues.	5–6% – Basic preprocessing is performed but important steps are missing.	0–4% – Preprocessing is inadequate, incorrect, or absent.
4. AI Technique Selection and Justification	10%	9–10% – Selects the most appropriate AI technique and provides strong technical justification based on the problem and dataset.	7–8% – Appropriate technique selected with reasonable justification.	5–6% – Technique is acceptable but justification is limited.	0–4% – Inappropriate technique or no meaningful justification.
5. Model Design / Architecture	10%	9–10% – Develops a clear, technically sound model architecture with appropriate components, parameters, and workflow.	7–8% – Model is well designed with minor omissions.	5–6% – Basic model design is presented but lacks technical depth.	0–4% – Model design is unclear, incomplete, or inappropriate.

6. Algorithm / Pseudocode	5%	5% – Algorithm/pseudocode is complete, logically correct, structured, and clearly represents the model development process.	4% – Mostly correct with minor omissions.	2–3% – Basic algorithm provided but contains gaps.	0–1% – Algorithm is missing or substantially incorrect.
7. Implementation and GitHub Repository	15%	13–15% – Fully functional implementation with clean, organized, reproducible code and a complete GitHub repository with README and supporting files.	10–12% – Functional implementation and well-organized repository with minor issues.	7–9% – Partially functional implementation or incomplete repository documentation.	0–6% – Implementation is incomplete/non-functional or GitHub repository is missing.
8. Experimental Design and Results	10%	9–10% – Conducts meaningful experiments with appropriate test cases and clearly presents results using tables, graphs, or visualizations.	7–8% – Appropriate experiments with clearly presented results.	5–6% – Limited experiments or basic result presentation.	0–4% – Insufficient, incorrect, or missing experimental results.
9. Performance Evaluation	10%	9–10% – Uses appropriate evaluation metrics and provides comprehensive quantitative analysis of model performance.	7–8% – Uses suitable metrics and provides adequate evaluation.	5–6% – Basic evaluation with limited metrics or analysis.	0–4% – Inappropriate metrics or performance evaluation is missing.
10. Result Analysis and Interpretation	5%	5% – Provides insightful analysis of results, identifies patterns, errors, strengths, weaknesses, and explains model behavior.	4% – Provides good interpretation with minor gaps.	2–3% – Basic interpretation with limited analysis.	0–1% – Results are not meaningfully analyzed.

11. Limitations and Future Enhancements	5%	5% – Clearly identifies realistic limitations and proposes technically relevant and feasible improvements.	4% – Identifies relevant limitations and reasonable improvements.	2–3% – Provides basic limitations and future suggestions.	0–1% – Missing, unrealistic, or poorly explained.
12. Documentation and Technical Presentation	10%	9–10% – Report is comprehensive, well organized, technically accurate, professionally presented, and properly referenced.	7–8% – Well documented with minor presentation or referencing issues.	5–6% – Adequate documentation with several gaps.	0–4% – Poorly documented, unclear, or incomplete report.
Total	100%				